Module: Models, ORM, and Django Admin

Theme apps: Blog · Task Manager · Booking · Student Portal · CampusCart E-commerce

Previous module: URLs Views Templates Static
Next module: Forms Sessions Auth

Module Outcome:

- Describe a table as a Python class and let Django talk SQL
- Create / list / update rows without writing raw INSERT in the view
- Use Admin to poke CampusCart data while you build the public pages

What is a Model?

A **model** is a Python class describing **database table structure**. Each **attribute** on the class becomes a **column**. Django **ORM** (Object-Relational Mapper) translates Python operations to SQL.

Where used?
- Campus portal — Student / Course models synced to MySQL
- **TechMart** — Product, Order models + Django admin for staff
- Blog — Post / Author / Tag with FK and M2M
- Task Manager — Task rows owned by User (FK later with auth)
- Capstone — replace hardcoded template lists with ORM queries

ORM — you work with Python objects; Django generates SQL like `SELECT * FROM blog_post WHERE ...`.

Migration — a version file describing schema changes (add column, new table). **`makemigrations`** creates the file; **`migrate`** applies it to MySQL.

Model Example (Blog)

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField()
    published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

Creates table `blog_post` (app label + model name lowercase).

Field Terms

Term	Meaning
`max_length`	Required for CharField — DB column size limit
`null=True`	Database allows NULL
`blank=True`	Form validation allows empty (admin/forms)
`default`	Value when not provided
`auto_now_add`	Set once on create (timestamp)

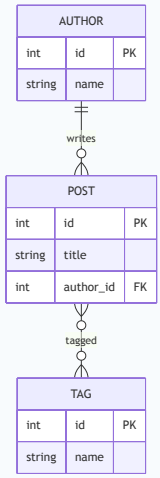
Note:
- Prefer `blank=True` only for optional text fields — avoid `null=True` on `CharField` unless you truly need SQL NULL (NULL vs empty-string confusion).

Relationships Overview

ForeignKey (FK) — **many-to-one**. Many blog posts → one author.
Example: `author = models.ForeignKey(Author, on_delete=models.CASCADE)`

ManyToManyField (M2M) — **many-to-many**. Post has many tags; tag on many posts. Django creates junction table automatically.

Visual — blog relationships:



Key takeaways:
- Author **1** → **many** Posts (ForeignKey); Post **many** ↔ **many** Tags (M2M)
- ORM = Python objects; migrations keep MySQL schema in sync
- Admin is the staff UI once models are registered

`on_delete=models.CASCADE` — if parent Author deleted, delete their posts too (common default for learning; discuss PROTECT in production).

Note:
- `ForeignKey` without `on_delete` → model error. Always set `on_delete` (e.g. `CASCADE`, `PROTECT`).

Django Admin

Built-in **staff interface** at `/admin/` for non-developers to manage data. Register models in **`admin.py`**. Requires **superuser** account.

Note:
- Forgot to register in `admin.py` → model missing from `/admin/`. Add `@admin.register(...)` or `admin.site.register(...)`.

Concepts

1. Migration Workflow

```
Change models.py → makemigrations → migrate → MySQL schema updated
```

Never edit applied migration files manually as a beginner — create new migration instead.

Note:
- Forgot `makemigrations` after editing `models.py` → no schema change. Run `makemigrations` then `migrate`.
- Editing an applied migration by hand often causes conflicts — regenerate in lab or ask in class.

Create and apply blog migrations, then list the blog app's migration status.

```
python manage.py makemigrations blog
python manage.py migrate
python manage.py showmigrations blog
```

Expected Output: (abbreviated)

```
Migrations for 'blog':
  blog/migrations/0001_initial.py
    - Create model Post
Applying blog.0001_initial... OK
 [X] 0001_initial
```

Open the generated migration and confirm CreateModel for Post matches `models.py` before you migrate.

```
# After makemigrations

Open `blog/migrations/0001_initial.py` and confirm:
- CreateModel for Post
- fields match models.py
Then migrate.
```

2. QuerySet

QuerySet — lazy collection of objects from database. `.filter(published=True)` returns QuerySet; DB hit may defer until iteration.

Note:
- `Post.objects.get()` with 0 or 2+ rows → `DoesNotExist` / `MultipleObjectsReturned`. Prefer `filter().first()` or handle the exception.
- N+1 queries in a template (looping related objects) slow the page — use `select_related('author')` when you know you'll need the FK (interview teaser).

In the Django shell, create a published Post and print titles of published posts.

```
# python manage.py shell
from blog.models import Post
Post.objects.create(title="Idli Special", body="₹40 at canteen", published=True)
print(list(Post.objects.filter(published=True).values_list("title", flat=True)))
```

Expected Output:

```
['Idli Special']
```

Remember: Change `models.py` → `makemigrations` → `migrate` — skipping either leaves MySQL out of sync.

Fetch the five newest posts with `order_by` and a slice, then print each id and title.

```
from blog.models import Post
latest = Post.objects.order_by("-id")[:5]
for p in latest:
    print(p.id, p.title)
```

3. `__str__` Method

Defines human-readable representation in admin and shell:

```
def __str__(self):
    return self.title
```

Give Post a `__str__` that returns the title so admin and the shell show a readable label.

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField()

    def __str__(self):
        return self.title
```

Give a TechMart Product a `__str__` that includes name and rupee price.

```
class Product(models.Model):
    name = models.CharField(max_length=120)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return f"{self.name} — ₹{self.price}"
```

4. Admin Customization (Intro)

```
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("title", "published", "created_at")
    list_filter = ("published",)
    search_fields = ("title",)
```

Register Post in admin with `list_display` and a title search.

```
# blog/admin.py
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("title", "published", "id")
    search_fields = ("title",)
```

Add a published sidebar filter on the Post admin changelist.

```
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("title", "published")
    list_filter = ("published",)
```

Step-by-Step Lab (Annotated)

Step 1 — Blog Models

```
# blog/models.py
from django.db import models

class Author(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)

    def __str__(self):
        return self.name

class Post(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField()
    published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    author = models.ForeignKey(Author, on_delete=models.CASCADE, related_name="posts")

    def __str__(self):
        return self.title

class Tag(models.Model):
    name = models.CharField(max_length=50, unique=True)
    posts = models.ManyToManyField(Post, related_name="tags", blank=True)

    def __str__(self):
        return self.name
```

Step 2 — Migrations

```
python manage.py makemigrations blog
python manage.py migrate
```

Verify in Workbench: tables `blog_author`, `blog_post`, `blog_tag`, `blog_tag_posts`.

Step 3 — ORM CRUD in Shell

```
python manage.py shell
```

```
from blog.models import Author, Post, Tag

# CREATE
a = Author.objects.create(name="Priya", email="priya@campus.in")
p = Post.objects.create(title="CampusCart Goes Live", body="Shop books online.", author=a, published=Tru
```

```
# READ
Post.objects.all()
Post.objects.filter(published=True)
Post.objects.get(pk=1)

# UPDATE
p.title = "CampusCart Launch Event"
p.save()

# DELETE
# Tag.objects.filter(name="news").delete()

# M2M
t, _ = Tag.objects.get_or_create(name="announcement")
p.tags.add(t)
```

Step 4 — Views Using ORM

```
# blog/views.py
from django.shortcuts import render
from .models import Post

def post_list(request):
    posts = Post.objects.filter(published=True).order_by("-created_at")
    return render(request, "blog/post_list.html", {"posts": posts})
```

```
<!-- blog/templates/blog/post_list.html excerpt -->
{% for post in posts %}
  <article>
    <h2>{{ post.title }}</h2>
    <p>By {{ post.author.name }} · {{ post.created_at|date:"d M Y" }}</p>
    <p>{{ post.body|truncatewords:30 }}</p>
  </article>
{% empty %}
  <p>No published posts.</p>
{% endfor %}
```

Step 5 — Task Manager Model

```
# tasks/models.py
from django.db import models

class Task(models.Model):
    PRIORITY_CHOICES = [
        ("L", "Low"),
        ("M", "Medium"),
        ("H", "High"),
    ]
    title = models.CharField(max_length=150)
    done = models.BooleanField(default=False)
    priority = models.CharField(max_length=1, choices=PRIORITY_CHOICES, default="M")
    due_date = models.DateField(null=True, blank=True)

    def __str__(self):
        return self.title
```

Step 6 — Admin Registration

```
python manage.py createsuperuser
# username: admin, email: admin@example.com, password: (lab standard)
```

```
# blog/admin.py
from django.contrib import admin
from .models import Author, Post, Tag

@admin.register(Author)
class AuthorAdmin(admin.ModelAdmin):
    search_fields = ("name", "email")

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("title", "author", "published", "created_at")
    list_filter = ("published", "author")
    search_fields = ("title", "body")

admin.site.register(Tag)
```

Visit `http://127.0.0.1:8000/admin/` — add authors, posts, tasks.

Step 7 — CampusCart Product Model (Preview)

```
# catalog/models.py (CampusCart app — create if capstone track)
class Product(models.Model):
    name = models.CharField(max_length=120)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)

    def __str__(self):
        return f"{self.name} (₹{self.price})"
```

Practice Exercises

Basic

1. Add `updated_at = models.DateTimeField(auto_now=True)` to Post; migrate
   Model answer:

```
updated_at = models.DateTimeField(auto_now=True)
```

`bash python manage.py makemigrations && python manage.py migrate`

2. Create 5 posts via admin; list only published in view
   Model answer:
   Register `Post` in admin, add five rows with `published=True/False`. In the view: `Post.objects.filter(published=True)`.
3. Task model: filter `done=False` in dashboard view
   Model answer:

```
tasks = Task.objects.filter(done=False).order_by("-created_at")
return render(request, "tasks/dashboard.html", {"tasks": tasks})
```

Intermediate

1. **Booking:** `Room` model (name, capacity, available) + admin list_filter on available
   Model answer:

```
class Room(models.Model):
    name = models.CharField(max_length=100)
    capacity = models.PositiveIntegerField()
    available = models.BooleanField(default=True)

@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_filter = ("available",)
```

2. **Student Portal:** `Student` FK to `Department` model
   Model answer:

```
class Department(models.Model):
    name = models.CharField(max_length=100)

class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.ForeignKey(Department, on_delete=models.CASCADE, related_name="students")
```

3. Shell: count posts per author using `author.posts.count()`
   Model answer:

```
from django.contrib.auth import get_user_model
User = get_user_model()
for u in User.objects.all():
    print(u.username, u.posts.count())
```

(Assumes `Post.author` FK with `related_name="posts"`.)

Interview Prep

1. Difference between `null=True` and `blank=True`
   Model answer:
   `null=True` allows SQL NULL in the database. `blank=True` allows empty values in forms/validation. CharFields usually use `blank=True` with `default=""` rather than NULL.
2. What happens on CASCADE delete?
   Model answer:
   Deleting the parent row also deletes related child rows that use `on_delete=models.CASCADE` (e.g. delete Department → its Students go too).
3. ORM vs raw SQL — when use each?
   Model answer:
   Prefer ORM for CRUD, safety, and portability. Use raw SQL for complex reports, vendor-specific features, or performance-critical queries the ORM expresses poorly — still parameterize inputs.

Mini-Project / Activity (Assignment)

Activity: "Admin-Driven Blog" (60 min)
Lab kit (preview ownership): 15_Lab_Assets/03_django_auth_tasks/

What to build

1. Models: Author, Post, Tag (M2M)
2. Admin customized with list_display and search
3. Public `/blog/` reads from DB
4. Add a post live in admin; refresh the page and confirm the new post appears

Stretch

- Add `published` Boolean and show only published posts on the public list.
- Add `DecimalField` price on a Product model for CampusCart preview (₹).

Done when

- [ ] Migrations applied; models visible in admin
- [ ] Public `/blog/` shows ORM data (no hardcoded posts)
- [ ] Admin `list_display` + search works
- [ ] New admin post appears after browser refresh

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. If ORM generates SQL anyway, why not write SQL directly?
2. Who uses Django admin in a real company — developers or operations staff?
3. CASCADE delete: safe for financial records? Alternatives?
4. How would CampusCart link `Order` to `Product` — FK or M2M?
5. compare Django models to JPA entities / Hibernate.

Q1. What is Django ORM?
A: API mapping Python classes to SQL tables; CRUD via QuerySets without writing most SQL manually.

Q2. makemigrations vs migrate?
A: makemigrations creates migration files from model changes; migrate applies them to database.

Q3. ForeignKey vs ManyToMany?
A: FK: many-to-one (many posts, one author). M2M: many-to-many (posts and tags) with junction table.

Q4. null vs blank?
A: null is database-level NULL allowed; blank is validation-level empty allowed in forms/admin.

Q5. Benefits of Django admin?
A: Rapid internal CRUD UI for staff; less custom code for back-office operations.

Q6. What is QuerySet?
A: Lazy database-backed collection of model instances; chainable filters before evaluation.